

HIGH-LEVEL DESIGN

User Posts & Albums Viewer

Client-side presentation-layer web application

Field	Value
Document Version	v0.3
Status	Draft - requirements clarification incorporated
Prepared By	Atul Sharma
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API

Revision Summary

Requirements clarification incorporated: Explicit independent Posts/Albums requests, simultaneous two-panel presentation, and presentation-only application boundary.

1. Purpose

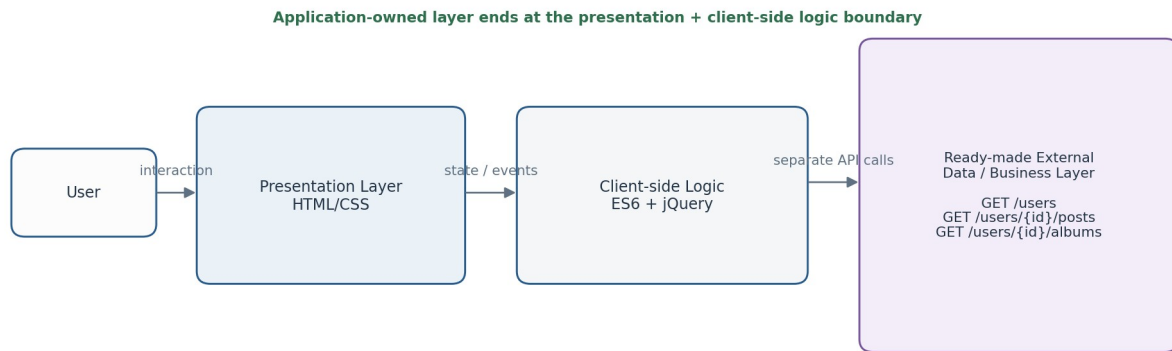
This HLD defines the major components, responsibilities, external interface, and high-level request/data flow for the User Posts & Albums Viewer. The application is intentionally limited to the presentation layer and consumes a ready-made external API/data layer.

2. Scope and Design Boundaries

- The solution is a client-side / presentation-layer web application.
- HTML and CSS provide page structure and presentation.
- JavaScript with jQuery provides interaction, API communication, response processing, state, and DOM updates.
- JSONPlaceholder provides the ready-made user, Post, and Album data layer used by the application.
- No custom backend, application-owned business/service layer, database, authentication, or persistent storage is introduced.

3. Architecture Overview

The browser contains the application-owned presentation and client-side logic. The external API provides the required data. After a user is selected, Posts and Albums are requested as two independent API calls and are rendered into two separate UI panels visible at the same time.

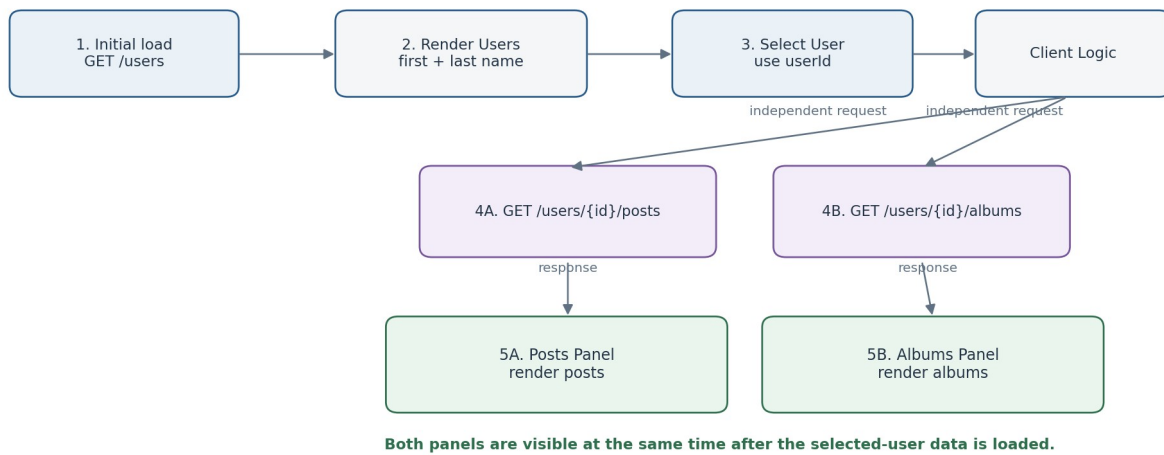
Figure 1. High-Level Architecture

4. Major Components

Component	Responsibility	Key technology
Presentation Layer	Renders users, selected-user area, Posts panel, Albums panel, loading, error, and empty states	HTML5 + CSS3
Client-side Application Logic	Handles user interaction, separate API requests, response processing, state, and DOM updates	JavaScript (ES6+) + jQuery 4.0.0
Ready-made External API/Data Layer	Supplies user, Post, and Album records	JSONPlaceholder

5. High-Level Data / Request Flow

1. Initial load: GET <https://jsonplaceholder.typicode.com/users>.
2. User rendering: render the returned name value as the requested first-name + last-name display.
3. User selection: capture the selected user ID.
4. Start two independent requests for the selected user: GET <https://jsonplaceholder.typicode.com/users/{userId}/posts> and GET <https://jsonplaceholder.typicode.com/users/{userId}/albums>.
5. These requests may execute concurrently because they are independent resources.
6. Render the returned Posts and Albums into separate panels.
7. Both panels remain visible at the same time.
8. Selecting another user clears the previous detail data and repeats the two requests.

Figure 2. Explicit Independent Request Flow

6. External API Interface

Purpose	Method	Complete URL	Used when
Retrieve users	GET	https://jsonplaceholder.typicode.com/users	Application initialization
Retrieve posts	GET	https://jsonplaceholder.typicode.com/users/{userId}/posts	After a user is selected
Retrieve albums	GET	https://jsonplaceholder.typicode.com/users/{userId}/albums	After a user is selected

7. Component Responsibilities and Interactions

7.1 Presentation Layer

- Displays the user list and selected user.
- Provides separate Posts and Albums sections that are both visible simultaneously.
- Displays loading, error, and empty feedback.

7.2 Client-side Application Logic

- Initiates independent Posts and Albums requests after selection.
- Tracks the selected user and request lifecycle.
- Processes JSON responses and updates the presentation layer.
- Prevents stale results from an earlier user selection from overwriting the current selection.

7.3 External API Boundary

- Provides user, Post, and Album data.
- No application-owned business service is inserted between the browser and the external API.

8. Interaction Scenarios

Scenario	Client-side flow	Result
Initial load	Initialize -> GET /users -> render users	Users visible
User selection	Capture ID -> GET posts + GET albums independently	Selected-user data retrieved
Posts rendering	Post response -> render Posts panel	Posts visible
Albums rendering	Album response -> render Albums panel	Albums visible
Simultaneous detail view	Two panels rendered together	Posts and Albums visible at the same time
API failure	Request failure -> error handling	User-facing error feedback

9. UI State Model

State	When it applies	Expected UI behavior
Initial / Users Loading	Initial GET /users	Show users loading indication
Users Loaded	Users successfully retrieved	Render user list
User Detail Loading	Posts and/or Albums are being retrieved	Show loading feedback in relevant detail sections
Posts Loaded	Posts response available	Render Posts panel
Albums Loaded	Albums response available	Render Albums panel
Error	A required request fails	Show meaningful error feedback in affected area
Empty	Successful response contains no items	Show explicit empty-state message in that panel

10. Cross-Cutting Design Considerations

10.1 Error Handling

- Non-success HTTP responses and network failures are treated as request failures.
- Because Posts and Albums are independent requests, failure handling can identify the affected resource.
- The UI should not remain in an unexplained loading state.

10.2 Loading and Feedback

Loading feedback is associated with the data currently being retrieved. Posts and Albums are separate resources, so their panel states may be tracked independently.

10.3 Maintainability

API communication, interaction/state handling, and DOM rendering remain logically separated. Endpoint construction uses the selected user ID rather than hard-coded user-specific URLs.

10.4 Security / Trust Boundary

API-returned text is treated as untrusted input and rendered using safe text-oriented DOM operations where practical. No authentication or authorization is included.

11. Performance Considerations

- Initial load retrieves only the users collection.
- User-specific requests are limited to the selected user.
- Posts and Albums may be requested concurrently because they are independent resources.

12. Deployment Assumption

The application is deployed as static/client-side assets and communicates directly with JSONPlaceholder over HTTPS.

13. HLD-to-LLD Boundary

The LLD defines exact files, functions, selectors, state fields, AJAX handling, loading/error markup, and rendering procedures.

14. Updated Design Decisions

- Presentation layer only; no application-owned business/service layer.
- Posts and Albums are separate API calls.
- Posts and Albums are rendered as separate panels and are visible simultaneously.
- JSONPlaceholder name is the display value for the requested first-name + last-name presentation.
- Client-side logic prevents stale user-detail responses from replacing current-user data.

15. Status

HLD v0.3 - Draft. This revision incorporates the latest requirements clarification about the presentation-only scope and simultaneous Posts + Albums display.